

[school-of-solana](#) / [3.lesson](#) / 



 **hyckomatej** Update materials

03c5a68 · 3 months ago 

Name	Name	Last commit date
 ..		
 calculator	Update materials	3 months ago
 README.md	Update materials	3 months ago

README.md



3. Lecture - Solana programming model I

Table of Contents

- [Solana Program Model](#)

- [Accounts](#)
- [Program types](#)
- [Entrypoint](#)
- [Anchor Framework](#)
 - [High-level Overview](#)
 - [The Accounts Struct](#)
 - [Types](#)
 - [The Account Type](#)
 - [Constraints](#)
 - [Safety checks](#)
 - [The Program Module](#)
 - [Context](#)
 - [Instruction Data](#)
 - [Errors](#)
 - [Anchor Internal Errors](#)
 - [Custom Errors](#)

Solana Program Model

Accounts

The essential building blocks of the Solana Programming model are the accounts. They can store either the data or programs, which splits them into two categories:

- **Executable accounts** - accounts that store native or user-created programs
- **Non-executable accounts** - accounts used for data storage

Accounts are identified by their address - **key** (a unique 32-byte `PubKey`). Each account has the **owner** (also of the `Pubkey` type), its **lamports** balance, boolean **executable** flag and **data** (where depending on the executable flag, either program or data is stored).

Note

Accounts can be reused, meaning that the account which was at one point used to store data can, after it was properly closed, be used again to store a program.

Program types

As we outlined in the accounts explanation, there are two types of programs:

- **Native Programs** - these are core Solana programs like System program, BPF Loader Program or Vote Program. They are owned by the Native Loader Program
- **On-chain Programs** - user-created programs deployed on Solana. They are owned by the BPF Loader

Note

Native programs are only updated as a part of migration from one Solana runtime version to another. On-chain programs can be updated by their upgrade authority.

Note

There are currently 3 types of BPF Loaders and only one of them allows you to upgrade the deployed program - `BPFLoaderUpgradeable1e`. Official docs are not always up-to-date on this, so if you want to see current number of existing BPF Loaders, see [source code](#) and look for `bpf_loader*`.

Entrypoint

Each solana program exports a known **entrypoint**, that is essentially a function which takes as an input a byte array containing the serialized program parameters (`program_id`, `accounts`, `instruction_data`).

Upon the `entrypoint` entry, the input byte array is deserialized into `program_id`, `accounts` and `instruction_data`. These are further passed to the user-defined `ProcessInstruction` function, where another deserialization of `instruction_data` takes place. Only then the instruction proceeds to corresponding user-defined actions.

Important

It can be a bit confusing to differentiate between the **transaction** and the **instruction**. The transaction consist of one or multiple instructions, and can be thought of as a package of instructions submitted to the network. The instruction is a call to the on-chain program via Solana Runtime that defines what program to call, which accounts the program will read/write to and the input to the program.

Anchor Framework

High-level Overview

An Anchor program consists of three parts. The **program module**, the **Accounts struct** which is marked with `#[derive(Accounts)]`, and the `declare_id` macro. The program module is where you write your business logic. The Accounts struct is where you validate accounts. The `declare_id` macro creates an ID field that stores the address of your program. Anchor uses this hardcoded ID for security checks and it also allows other crates to access your program's address.

When you start up a new Anchor project, you'll see the following:

```
// use this import to gain access to common anchor features
use anchor_lang::prelude::*;

// declare an id for your program
declare_id!("Fg6PaFpoGXkYsidMpWTK6W2BeZ7FEfcYkg476zPFsLnS");

// write your business logic here
#[program]
mod hello_anchor {
    use super::*;
    pub fn initialize(_ctx: Context<Initialize>) -> Result<()> {
        Ok(())
    }
}

// validate incoming accounts here
```



```
#[derive(Accounts)]  
pub struct Initialize {}
```

We'll go into more detail in the next sections but for now, note that the way an endpoint is connected to its corresponding **Accounts struct** is the `ctx` argument in the endpoint. The argument is of type `Context` which is generic over an **Accounts struct**, i.e. this is where you put the name of your account validation struct. In this example, it's `Initialize`.

Important

We ended the previous section with the talk on the [entrypoint](#). Programs written in Anchor also have the entrypoint but it is automatically created for us with the `program` attribute macro. Thanks to this, we can focus purely on the writing of the instructions logic.

The Accounts Struct

The **Accounts struct** is where you define which accounts your instruction expects and which constraints these accounts should adhere to. You do this via two constructs: Types and constraints.

Types

Tip

[Account Types Reference](#)

Each type has a specific use case. Detailed explanations for the types can be found in the Account Types Reference above. We will briefly explain the most important type here, the `Account` type.

The Account Type



Tip

[Account Reference](#)

The `Account` type is used when an instruction is interested in the deserialized data of the account. Consider the following example where we set some data in an account:

```
// use this import to gain access to common anchor features
use anchor_lang::prelude::*;

// declare an id for your program
declare_id!("Fg6PaFpoGXkYsidMpWTK6W2BeZ7FEfcYkg476zPFsLnS");

// write your business logic here
#[program]
mod hello_anchor {
    use super::*;
    pub fn set_data(ctx: Context<SetData>, data: u64) -> Result<()> {
        ctx.accounts.my_account.data = data;
        Ok(())
    }
}

// define data you want to store, here
#[account]
#[derive(Default)] // this derives default values for the struct fields
pub struct MyAccount {
    data: u64
}
```



```
// validate incoming accounts here
#[derive(Accounts)]
pub struct SetData<'info> {
    #[account(mut)]
    pub my_account: Account<'info, MyAccount>
}
```

`Account` is generic over `T`. This `T` is a type you can create yourself to store data. In this example, we have created a struct `MyAccount` with a single data field to store a `u64`. `Account` requires `T` to implement certain functions (e.g. functions that (de)serialize `T`). Most of the time, you can use the `#[account]` attribute to add these functions to your data, as is done in the example.

Most importantly, the `#[account]` attribute sets the owner of that data to the `ID` (the one we created earlier with `declare_id`) of the crate `#[account]` is used in. The `Account` type can then check for you that the `AccountInfo` passed into your instruction has its owner field set to the correct program. In this example, `MyAccount` is declared in our own crate so `Account` will verify that the owner of `my_account` equals to the address we declared with `declare_id`.

Constraints

 Tip

[Constraints reference](#)

`Account` type can do a lot of work for you but it is not dynamic enough to handle all the security checks a secure program requires.

Add constraints to an account with the following format:


```
#[account(<constraints>)]  
pub account: AccountType
```



Some constraints support custom Errors (we will explore errors later):

```
#[account(...,<constraint> @ MyError::MyErrorVariant, ...)]  
pub account: AccountType
```



For instance, in the examples below, we used the `mut` constraint to indicate that `my_account` should be mutable. We used `has_one` to check that the `token_account.owner == owner.key()`. And finally we used `constraint` to check an arbitrary expression; in this case, whether the incoming `TokenAccount` belongs to the `my_account.mint`.

```
// Defines the context for the `set_data` function.  
#[derive(Accounts)]  
pub struct SetData<'info> {  
    // Mutable account of type `MyAccount`.  
    #[account(mut)]  
    pub my_account: Account<'info, MyAccount>,  
  
    // Validates that `my_account` and `token_account` share the same mint and links to `owner`.  
    #[account(  
        constraint = my_account.mint == token_account.mint, // Checks mint equality.  
        has_one = owner // Links to the owner's signature.  
    )]  
    pub token_account: Account<'info, TokenAccount>,  
  
    // Signer of the transaction, confirming authorization.
```



```
pub owner: Signer<'info>  
}
```

Safety checks

Two of the Anchor account types, `AccountInfo` and `UncheckedAccount` do not implement any checks on the account being passed. Anchor implements safety checks that encourage additional documentation describing why additional checks are not necessary.

Attempting to build a program containing the following excerpt with `anchor build`:

```
#[derive(Accounts)]  
pub struct Initialize<'info> {  
    pub potentially_dangerous: UncheckedAccount<'info>  
}
```



will result in an error similar to the following:

Error:

```
/anchor/tests/unchecked/programs/unchecked/src/lib.rs:15:8  
Struct field "potentially_dangerous" is unsafe, but is not documented.  
Please add a `/// CHECK:` doc comment explaining why no checks through types are necessary.  
See https://book.anchor-lang.com/anchor\_in\_depth/the\_accounts\_struct.html#safety-checks for more  
information.
```



To fix this, write a doc comment describing the potential security implications, e.g.:

```
#[derive(Accounts)]
pub struct Initialize<'info> {
    /// CHECK: This is not dangerous because we don't read or write from this account
    pub potentially_dangerous: UncheckedAccount<'info>
}
```



The Program Module

The program module is where you define your business logic. You do so by writing functions which can be called by clients or other programs. You've already seen one example of such a function, the `set_data` function from the previous section.

```
#[program]
mod hello_anchor {
    use super::*;
    pub fn set_data(ctx: Context<SetData>, data: u64) -> Result<()> {
        // Check if `token_account` has more than 0 tokens before proceeding.
        if ctx.accounts.token_account.amount > 0 {
            // If the condition is met, assign the provided `data` to `my_account.data`.
            ctx.accounts.my_account.data = data;
        }
        // Return success regardless of whether data was set or not.
        Ok(())
    }
}
```



Context



Tip

[Context Reference](#)

Each endpoint function takes a `Context` type as its first argument. Through this context argument it can access the `accounts` (`ctx.accounts`), the `program id` (`ctx.program_id`) of the executing program, `bumps` of the included accounts and the `remaining accounts` (`ctx.remaining_accounts`). The `remaining_accounts` is a vector that contains all accounts that were passed into the instruction but are not declared in the `Accounts` struct. This is useful when you want your function to handle a variable amount of accounts, e.g. when initializing a game with a variable number of players.

Instruction Data

If your function requires instruction data, you can add it by adding arguments to the function after the context argument. Anchor will then automatically deserialize the instruction data into the arguments. You can have as many as you like. You can even pass in your own types as long as you use `#[derive(AnchorDeserialize)]` on them or implement `AnchorDeserialize` for them yourself. Here's an example with a custom type used as an instruction data argument:

```
#[program]
mod hello_anchor {
  use super::*;

  // Updates `data` and `age` fields in `my_account` based on the provided `Data`.
  pub fn set_data(ctx: Context<SetData>, data: Data) -> Result<> {
    // Set `data` in `my_account` from the input `data` struct.
    ctx.accounts.my_account.data = data.data;
    // Set `age` in `my_account` from the input `data` struct.
    ctx.accounts.my_account.age = data.age;
    Ok(())
  }
}
```



```

}

#[account]
#[derive(Default)]
pub struct MyAccount {
    pub data: u64,
    pub age: u8
}

// Data struct used as an input for instructions, serializable and deserializable.
#[derive(AnchorSerialize, AnchorDeserialize, Eq, PartialEq, Clone, Copy, Debug)]
pub struct Data {
    pub data: u64,
    pub age: u8
}

```

Conveniently, `#[account]` implements `Anchor(De)Serialize` for `MyAccount`, so the example above can be simplified.

```

#[program]
mod hello_anchor {
    use super::*;
    pub fn set_data(ctx: Context<SetData>, data: MyAccount) -> Result<()> {
        // Replace the inner state of `my_account` with the provided `data`.
        ctx.accounts.my_account.set_inner(data);
        Ok(())
    }
}

```

```

#[account]

```



```
#[derive(Default)]
pub struct MyAccount {
    pub data: u64,
    pub age: u8
}
```

Errors

There are two types of errors in anchor programs. `AnchorErrors` and non-anchor errors. `AnchorErrors` can be divided into `Anchor Internal Errors` that the framework returns from inside its own code or `Custom errors` which the user (you!) can return. Non-anchor are those errors thrown by the Solana Runtime (e.g. `UNINITIALIZED_ACCOUNT` or `INVALID_SEEDS` errors).

- `AnchorErrors`
 - `Anchor Internal Errors`
 - `Custom Errors`
- Non-anchor errors

Anchor Internal Errors



Tip

[Anchor Internal Error Code Reference](#)

Anchor has many different internal error codes. These are not meant to be used by users, but it's useful to study the reference to learn about the mappings between codes and their causes. They are, for example, thrown when a constraint has been violated, e.g. when an account is marked with `mut` but its `is_writable` property is `false`.

Custom Errors

You can add errors that are unique to your program by using the `error_code` attribute.

Simply add it to an enum with a name of your choice. You can then use the variants of the enum as errors in your program. Additionally, you can add a message attribute to the individual variants. Clients will then display this error message if the error occurs. Custom Error code numbers start at the [custom error offset](#).

To actually throw an error use the `err!` macro. These add the file and line information to the error that is then logged by anchor.

```
#[program]
mod hello_anchor {
  use super::*;
  pub fn set_data(ctx: Context<SetData>, data: MyAccount) -> Result<()> {
    // Validate that `data` is less than 100, returning an error if it's not.
    if data.data >= 100 {
      return err!(MyError::DataTooLarge);
    }
    ctx.accounts.my_account.set_inner(data);
    Ok(())
  }
}

// Defines custom errors.
#[error_code]
pub enum MyError {
  // Error when data exceeds the allowed maximum value.
  #[msg("MyAccount may only hold data below 100")]

```



```
    DataTooLarge
}
```

require!

You can use the `require!` macro to simplify writing errors. The code above can be simplified to this (Note that the `>=` flips to `<`):

```
#[program]
mod hello_anchor {
    use super::*;
    pub fn set_data(ctx: Context<SetData>, data: MyAccount) -> Result<()> {
        // Ensures `data` is below 100, triggers an error if not.
        require!(data.data < 100, MyError::DataTooLarge);
        ctx.accounts.my_account.set_inner(data);
        Ok(())
    }
}

// Defines custom errors
#[error_code]
pub enum MyError {
    // Error when data exceeds the allowed maximum value.
    #[msg("MyAccount may only hold data below 100")]
    DataTooLarge
}
```



 Tip

There are a couple of `require!` macros to choose from ([search for require in the docs](#)). When comparing public keys, it's important to use the keys variants of the require statements like `require_keys_eq` instead of `require_eq` because comparing public keys with `require_eq` is very expensive.

Additional Resources

Tip

Check out additional resources for more information:

- [The Solana Programming Model by Helius](#)
- [Solana Documentation](#)
- [Anchor Documentation](#)

Need help?

If you have any questions feel free to reach out to us on [Discord](#).